

Sovereign Mode: verifiable execution for institutional AI

The method for running vendor AI products under institutional authority over data, model access, and the evidence of every session.

PUBLISHER

Ember Sovereignty Systems

VERSION

0.2 · draft for review

DATE

July 2026

CONTACT

embersovereignty.com/briefing

ABSTRACT

Institutional adoption of AI products stalls on a structural question: who holds authority over data once an authenticated session enters a vendor’s application? Contracts, zero-data-retention terms, and enterprise SSO govern important edges of the relationship, but they do not independently prove what code ran, which provider received a prompt, or what authority an agent exercised inside the session.

Sovereign Mode runs vendor application logic inside a sealed, hardware-attested environment. Institutional content is encrypted on the user’s device, decrypted only inside that environment, and never persisted as plaintext. Institutional policy governs the code, venue, and actions used in the session. Every session produces a signed receipt that a third party verifies without trusting the vendor or Ember.

CONTENTS

1 The institutional problem and the parties

2 Method properties

3 Threat model and trust assumptions

4 The sealed execution environment

5 Code identity and attestation

6 The control point: an attested model-call router

7 Identity, authority, and credentials

8 Policy

9 Receipts

10 Sovereign Mode: a full session

11 Governance and upgrades

12 Deployment models and assurance

13 Operational model

14 Boundaries and open problems

15 Conclusion

A Appendix: verifying an attestation

B Appendix: receipt field reference

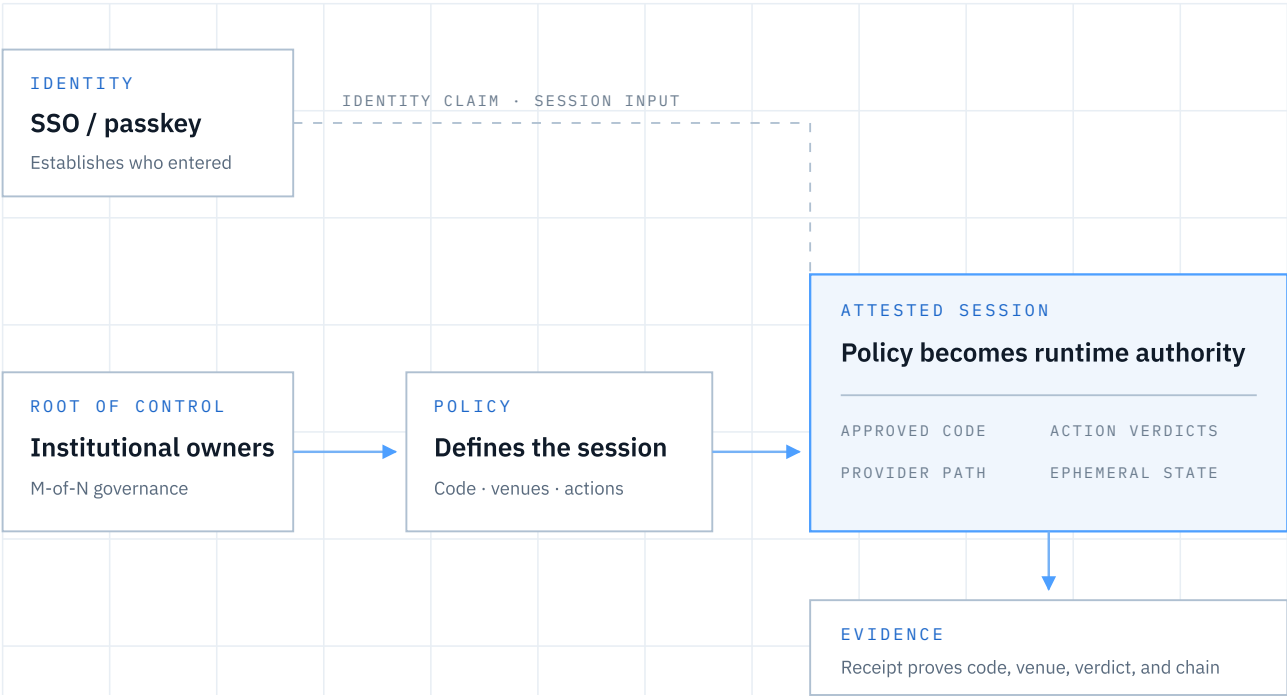
C Glossary

D Selected references

Institutional authority persists through the session.

Identity establishes who entered. Institutional policy governs what that identity authorizes, which systems receive data, and what evidence the session leaves behind.

IDENTITY ENTERS · AUTHORITY CONTINUES



1. The institutional problem and the parties

An AI product is only institution-ready if it can answer one question with evidence: **who holds authority over our data while the product works on it?**

1.1 The parties

This paper uses four nouns deliberately. The **institution** is the governed organization whose data, policy, and audit obligations are at issue. The **vendor** builds and operates the AI product. The **inference provider** operates a model endpoint. A **user** is a person or agent acting for the institution. In commercial settings the institution is often the vendor's enterprise buyer; calling every party a "customer" obscures who actually controls each layer.

1.2 The problem

Modern AI products are built to read everything. Documents, repositories, deal rooms, and records flow into the vendor's service as plaintext, where retrieval pipelines, prompt assembly, and agents operate before and after each model call. The vendor's answer to the security question is contractual: no training on institutional data, zero data retention, access restricted to named roles. These commitments matter, but they remain **policy**. The architecture does not prevent the vendor, an attacker inside the vendor, a compelled disclosure, or a future acquirer from reading the data. A reviewer can audit the policy; they cannot verify the property.

The buyer's traditional alternatives trade one problem for another. On-premises and bring-your-own-cloud (BYOC) deployments move the vendor's code inside the buyer's perimeter, but the data remains readable there, the perimeter must be built and maintained by the buyer, and the vendor's release pipeline still ships unverified code into it. Section 12 treats this comparison structurally.

The result is a familiar stall: the product wins the evaluation and loses months in security review, and the most regulated (and highest-paying) institutions are exactly the ones who cannot proceed.

1.3 The claim, stated precisely

Sovereign Mode is the clean-room product profile: it runs an existing AI product under institutional authority. In every Sovereign Mode session, the following holds:

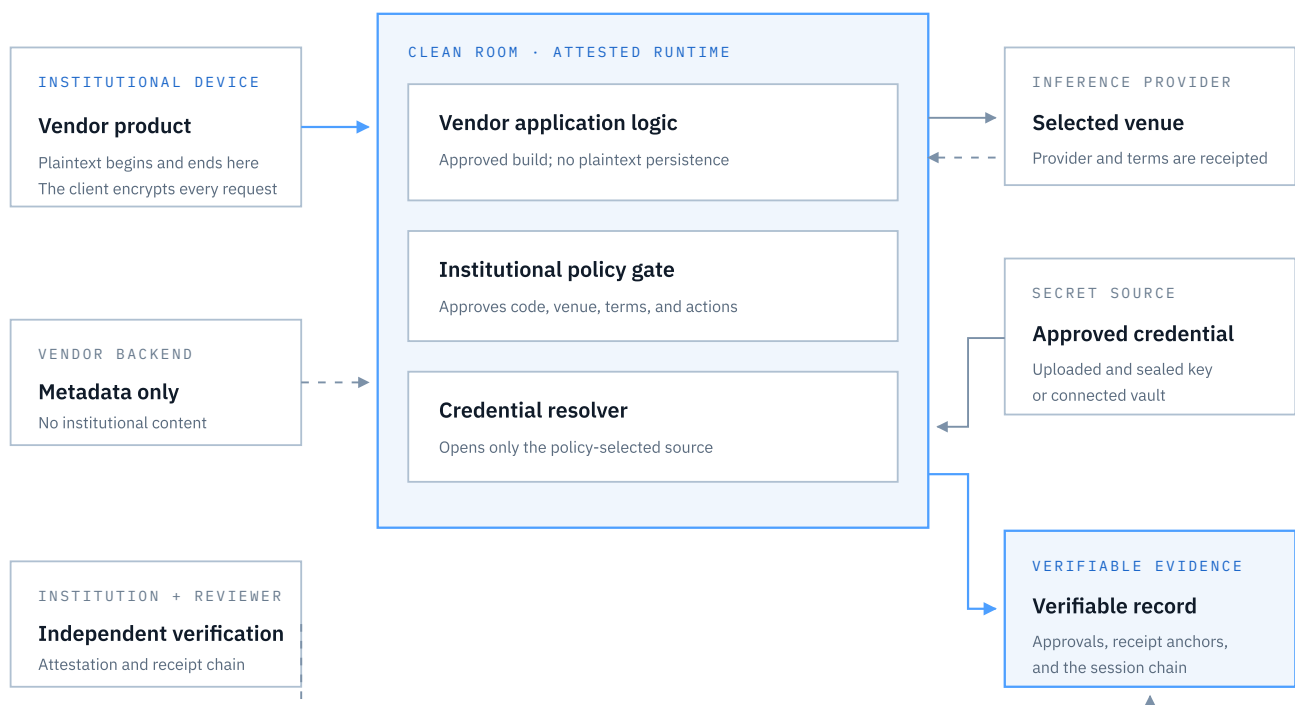
- The vendor's application logic (prompt assembly, retrieval, agent steps) runs unchanged, but inside a **sealed, hardware-attested execution environment** whose code identity is pinned by content hash and approved on an append-only ledger any reviewer can inspect.
- Institutional content is **encrypted on the user's device** to that specific attested environment. It is decrypted only inside it, and no plaintext session state persists.
- Calls to inference providers are made **from inside the environment**, using an institution- or vendor-controlled credential opened from its approved sealed or connected source. Institutional policy selects the permitted source, provider, endpoint, and terms.
- Every session emits a **signed receipt**: what code ran, where it ran, which credential class and provider were used, and which policy verdicts governed the call. Receipts chain, and any third party can verify them without

trusting the vendor or Ember.

- The vendor’s servers receive **session metadata only**. The vendor serves the entire product and can read none of the content.

The routing and governed-tools rungs in Section 12 are narrower control-point integration profiles. They use the same authority and evidence model, but they do not claim client-side encryption or vendor blindness. At every rung, a receipt certifies only the properties the session actually earned.

FIG. 1 System overview at the clean-room profile. Content is readable in exactly two places: the user-facing app and the attested clean room. The vendor backend coordinates on metadata; the ledger carries approvals, policy commitments, and receipt anchors. At the base-URL entry profile, only the control-point functions are attested; the vendor application moves inside at rung 3.



1.4 The pieces

The system is built from seven components. Each is introduced here and specified in its own section.

The clean room (§4). A sealed execution environment with memory isolation and remote attestation, available on verified third-party compute or institution-operated hardware. It runs the vendor’s logic and holds session plaintext; no plaintext session state persists after the session ends.

Code identity (§5). Every program the clean room runs is pinned by a content identifier (CID), a cryptographic hash of the exact build. The environment refuses code whose hash is not approved on the ledger, and its attestation quotes the hash of what is actually running.

The control point (§6). An attested model-call router. Any AI SDK or agent harness that speaks a supported wire format points at it with a base URL and gains policy-bound credential resolution, venue enforcement, and per-call receipts without a harness rewrite. It is both the integration on-ramp and the egress chokepoint of the clean room.

Authority and credentials (§7). Institutions hold the root of session authority through a passkey-controlled account with an M-of-N owner set. Institution- or vendor-controlled provider credentials enter through an uploaded-key path or a connected secret manager and open only to approved, attested code. Institutional authority over routing and session policy does not depend on which party funds inference; provider-side handling remains under the selected provider agreement until model execution is attested.

Policy (§8). A deterministic gate evaluated before any egress: which providers and endpoints are allowed, under which terms, within which budgets. Verdicts are `allow`, `allow-with-notify`, `hold` (pending human approval), or `deny`, and each verdict is recorded in the receipt.

Receipts (§9). A signed record per call and per session, produced inside the environment: code hash, venue, credential-owner class and fingerprint, content hashes, policy verdict, duration. Receipts chain to one another and verify independently.

Governance (§11). Code upgrades pass an on-chain M-of-N approval before the environment loads them. Tenant organizations begin under vendor administration and transfer to institutional ownership without a migration.

1.5 Two integration surfaces, kept distinct

Two different kinds of software touch this system, and the distinction matters throughout the paper. An **agent harness** is third-party software the vendor already runs: an AI SDK, a coding agent, or a custom multi-model loop. Harnesses are never modified; they are pointed at the control point by changing a base URL. The **Ember SDK** is the component a vendor embeds in its own user interface to deliver full Sovereign Mode: it verifies attestation, encrypts content client-side, and renders the sovereignty state and receipts to the end user. The control point requires no SDK at all; the SDK is how the complete product experience (Section 10) is delivered.

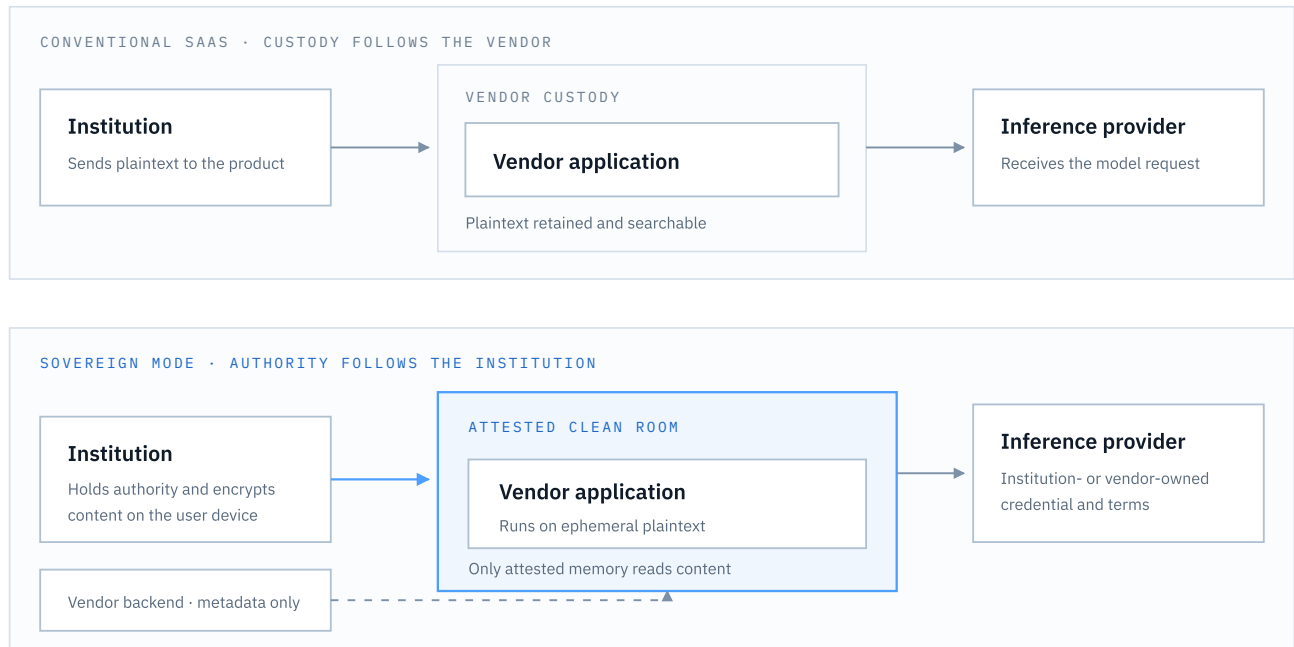
2. Method properties

The full clean-room profile provides six security properties and one adoption property. Narrower control-point profiles provide the named subset identified in Section 12. Each claim is verifiable; later sections identify the mechanism that enforces it.

-
- | | |
|----|--|
| G1 | Operator blindness. The vendor operating the product, and Ember operating the control plane, cannot read institutional content at any point in the processing path. Confidentiality holds against the operators themselves, not only outsiders. |
|----|--|
-
- | | |
|----|--|
| G2 | Code integrity. Only builds that have been pinned by content hash and approved on the ledger can execute in the environment. The running code's identity is quoted in hardware attestation, so "what was audited is what ran" is checkable per session. |
|----|--|
-
- | | |
|----|---|
| G3 | Institutional authority. The root of control is an account the institution governs, with M-of-N ownership for organizational use. No vendor-side or Ember-side credential can loosen institutional policy or grant access to content. Provider credentials are resources below this authority, not substitutes for it. |
|----|---|
-
- | | |
|----|---|
| G4 | Policy-governed egress. Nothing leaves the environment except through a fail-closed gate that evaluates the institution's policy: allowed providers, endpoints, terms, and budgets. Routing decisions are the institution's, provably. |
|----|---|
-
- | | |
|----|---|
| G5 | Independent verifiability. Every claim above is checkable by a third party (a security reviewer, an auditor, a regulator) using the returned receipt, the attestation quote, and the ledger, without trusting the vendor or Ember, holding an Ember account, or obtaining an operator log. |
|----|---|
-
- | | |
|----|---|
| G6 | Provider relationship preserved. Inference calls authenticate with an institution- or vendor-controlled credential supplied through the uploaded-key path or a connected secret manager. The credential owner's existing terms, including negotiated zero-data-retention and pricing, apply unchanged; the source class and key fingerprint are bound into the receipt. The receipt proves which relationship was selected, not the provider's compliance with its contract. |
|----|---|
-
- | | |
|----|--|
| G7 | No product rebuild. The vendor's prompts, retrieval, and agents run unchanged; end-user experience is unchanged. Routing, tools, and clean-room profiles retain external frontier models; attested inference depends on model artifacts available in an approved confidential venue. Adoption starts at a base URL and grows to the full SDK integration. A property that requires a rewrite does not qualify for deployment, so this constraint is load-bearing. |
|----|--|
-

Explicit boundaries and open problems, including side channels, availability guarantees, and the trust placed in the baseline vendor frontend, are collected in Section 14 rather than footnoted away.

FIG. 2 Custody, before and after. The same parties and the same product. In Sovereign Mode the vendor application moves inside the attested boundary, institutional policy becomes the root of session authority, and inference funding remains an independent choice.



3. Threat model and trust assumptions

A security architecture is defined by what it assumes as much as by what it prevents. This section states both, before any mechanism is described.

3.1 Adversaries considered

- A1 — The vendor’s infrastructure, honest-but-curious or compromised.** The primary adversary. It covers the vendor’s ordinary operation (logs, analytics, debugging), a breach of the vendor, an insider, a subpoena or other compelled disclosure served on the vendor, and a future acquirer inheriting the vendor’s data estate. Against A1, Sovereign Mode collapses custody: there is nothing readable to breach, compel, or acquire.
- A2 — The infrastructure operator, including Ember.** Whoever runs the control plane and the hosts under the enclaves: Ember in the hosted venue, a cloud operator, or the institution’s own platform team in self-hosted venues. A2 controls the operating system, the network, and scheduling. It must not be able to read session content, extract resolved credentials or sealed secrets, or forge receipts.
- A3 — A network attacker.** Standard Dolev-Yao capability on every link: observe, replay, reorder, and inject traffic between the client, the environment, providers, and the ledger.
- A4 — A malicious upgrade.** An attempt, by the vendor or by whoever holds its release pipeline, to ship code into the environment that leaks content while presenting a compliant surface. Handled by the governance gate of Section 11.
- A5 — A dishonest operator of the receipt log.** An attempt to rewrite history: dropping, reordering, or altering receipts after the fact. Handled by in-enclave signing, hash chaining, and anchoring to the ledger (Section 9).

3.2 What is trusted

The trusted base is deliberately short, and each item is named so a reviewer can price it:

- **The silicon vendor’s attestation root.** Hardware attestation is the foundation: the CPU manufacturer’s key signs the statement “this measurement is running inside a genuine enclave.” This is the same root of trust that confidential-computing deployments at the major clouds rest on.
- **The pinned, audited build.** Attestation proves which code ran; it does not make that code correct. The build must be audited (by the vendor, the institution, or a third party), and the audit is meaningful because the hash pins exactly what was reviewed.
- **The institution’s endpoint and authority credentials.** Content is encrypted on the endpoint; a compromised device reads content before encryption. Endpoint security and protection of the institution’s owner credentials remain institutional responsibilities.
- **The inference provider, to the extent policy admits it.** A prompt routed to an external model provider is plaintext to that provider, under the terms of the institution- or vendor-owned credential that authenticated it (Section 6). The system makes the credential source, routing, and terms provable; it does not make an external provider blind. Attested inference is a stronger assurance profile described in Section 12.

- **Liveness of the ledger** used for approvals and anchoring. Safety does not depend on it minute-to-minute; auditability does.

3.3 What is explicitly not assumed

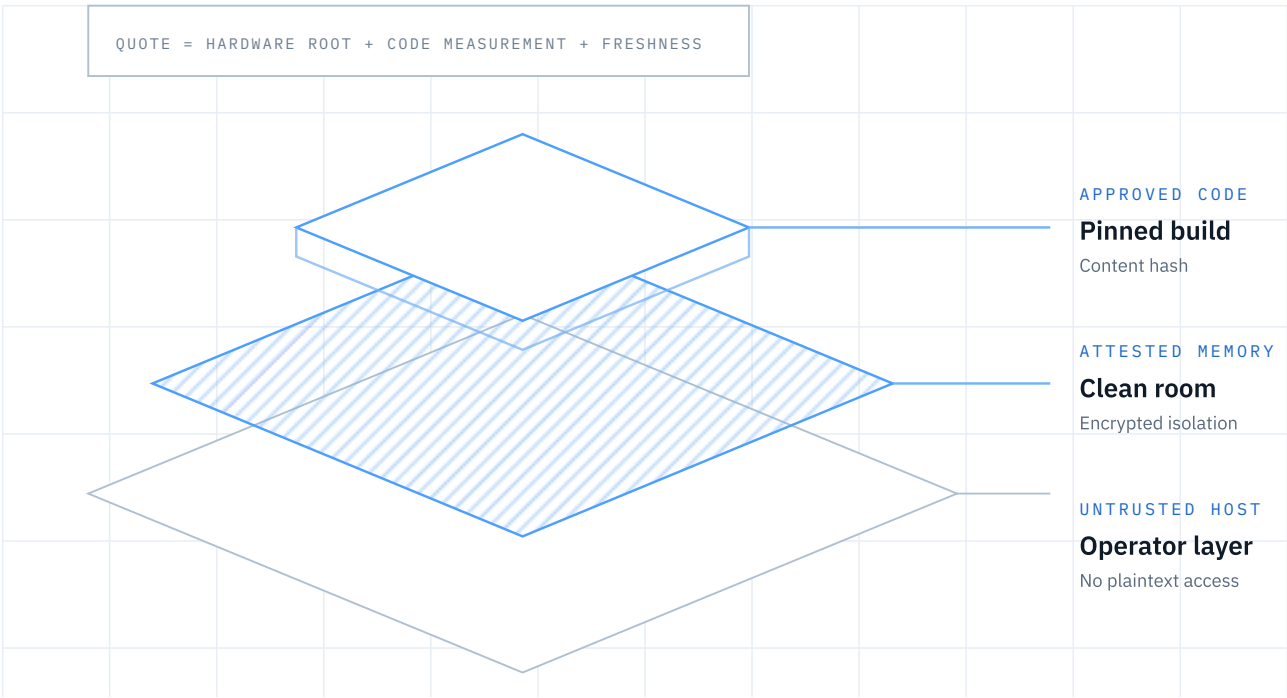
No trust is placed in the vendor's backend, the vendor's release pipeline (past the governance gate), Ember's organization or employees, the host operating systems under the enclaves, or the network. None of these can read content, use a resolved provider credential outside approved execution, or produce a receipt that verifies.

Baseline profile: the vendor-authored *frontend* on the user's device is trusted to verify attestation and encrypt honestly. This profile defends custody collapse against A1–A5; it does not defend against a vendor that deliberately ships a malicious client. Isolated trusted surfaces and reproducible client builds define stronger client profiles in Section 14.

Third-party compute is independently verifiable.

Attestation binds each workload to named hardware roots and an approved code measurement. Assurance follows the evidence, not the operator’s brand.

THE VERIFIED COMPUTE STACK



4. The sealed execution environment

The clean room is a trusted execution environment (TEE): a hardware-isolated context whose memory is encrypted by the CPU and inaccessible to the host operating system and hypervisor under the platform's threat model. Remote attestation follows the evidence, appraisal, and relying-party pattern standardized by the IETF RATS architecture.^[3] Two hardware properties carry the method:

- **Memory encryption and isolation.** While a session runs, its plaintext exists only inside the enclave's encrypted memory. The operator of the machine (A2) sees ciphertext pages and cannot introspect the workload.
- **Remote attestation.** The CPU signs a *quote*: a statement, rooted in the silicon vendor's key, of exactly what code and configuration booted inside the enclave. A remote party verifies the quote against the manufacturer's published root of trust before sending a single byte, establishing that genuine attested hardware is running the approved build.

Ember requires no bespoke fleet. The clean room runs on a network of attested TEE nodes, inside an institution's cloud account, or on institution-owned hardware. In every venue the runtime publishes attestation, resolves approved code by content hash, and refuses an unapproved measurement (Section 5). The reviewer evaluates evidence rather than the operator's brand.

4.1 Session lifecycle

A session is the unit of confidentiality. Its lifecycle is: attest, then encrypt to the attestation, then execute, then evaporate.

1. **Attest.** The client (the Ember SDK inside the vendor's app) fetches the environment's attestation quote and verifies it: genuine hardware, approved measurement (Section 5), fresh.
2. **Encrypt to the attestation.** The client encrypts content to a key that exists only inside the attested environment. Encryption is bound to the verified measurement, so content cannot be decrypted by a different build, a different machine, or a simulator.
3. **Execute.** The pinned vendor logic runs. Model calls leave only through the policy gate (Section 8) under an approved credential resolved as described in Section 7. Results are encrypted back to the client.
4. **Evaporate.** Session state is not persisted. Durable artifacts are exactly two: the receipt (metadata and hashes, no content) and, if the product uses it, memory encrypted under the institution's key, readable next session only by the same attested code path.

4.2 Venue independence

The guarantees are properties of the attested workload, not of who owns the building. The same pinned build, credential boundary, policy, and receipt format operate on hosted compute, inside the institution's cloud account, or on hardware the institution owns. Moving venue changes latency, procurement, and operator control; it does not remove the verification procedure. Section 12 places these choices on the assurance continuum.

5. Code identity and attestation

Attestation answers “is this genuine sealed hardware, and what booted in it?” The remaining question is whether *what booted* is *what was audited*. The system closes that gap with content addressing.

5.1 Content pinning

Every program the environment runs (the vendor’s application logic, and the control-point router itself) is identified by a content identifier (CID): a cryptographic hash over the exact bytes of the build. The CID is registered on the ledger before the code loads, and registration is guarded by the M-of-N approval described in Section 11. At load time the environment recomputes the hash of the code it was handed; a mismatch with a registered, approved CID is an integrity violation and the load is refused. There is no path to run improvised code, patched code, or a “debug build” in a clean room.

Pinning makes an audit portable. A reviewer who audits build `bafy...9k3q` is not auditing a release process or a promise; they are auditing bytes whose hash appears, verbatim, in every attestation quote and every receipt produced by that code.

5.2 The reviewer’s three checks

Before the first institutional payload enters the environment, the institution’s security team verifies it independently. Three checks require no trust in the vendor or Ember:

1. **Pull the attestation quote.** Every sovereign session exposes one through its verification endpoint.
2. **Verify the quote’s signature** against the silicon vendor’s published root of trust. This proves genuine attested hardware, not an emulator or a mock.
3. **Compare the measurement** in the quote to the CID of the audited build, and to the session’s receipt. Equal hashes prove the audited code is exactly what ran.

Appendix A walks through the protocol-level fields rather than prescribing a product endpoint. The receipt chain repeats these facts for every subsequent session, so drift is visible immediately: what a reviewer audits once, the architecture re-proves continuously.

6. The control point: an attested model-call router

Model calls are the shared boundary across otherwise different AI products: prompts, retrieved documents, tool results, and intermediate state all reach an inference endpoint through a small number of wire protocols. That boundary is therefore a practical place to enforce credential resolution, routing, policy, and evidence without rewriting the surrounding harness. The control point sits there.

6.1 Integration: a base URL

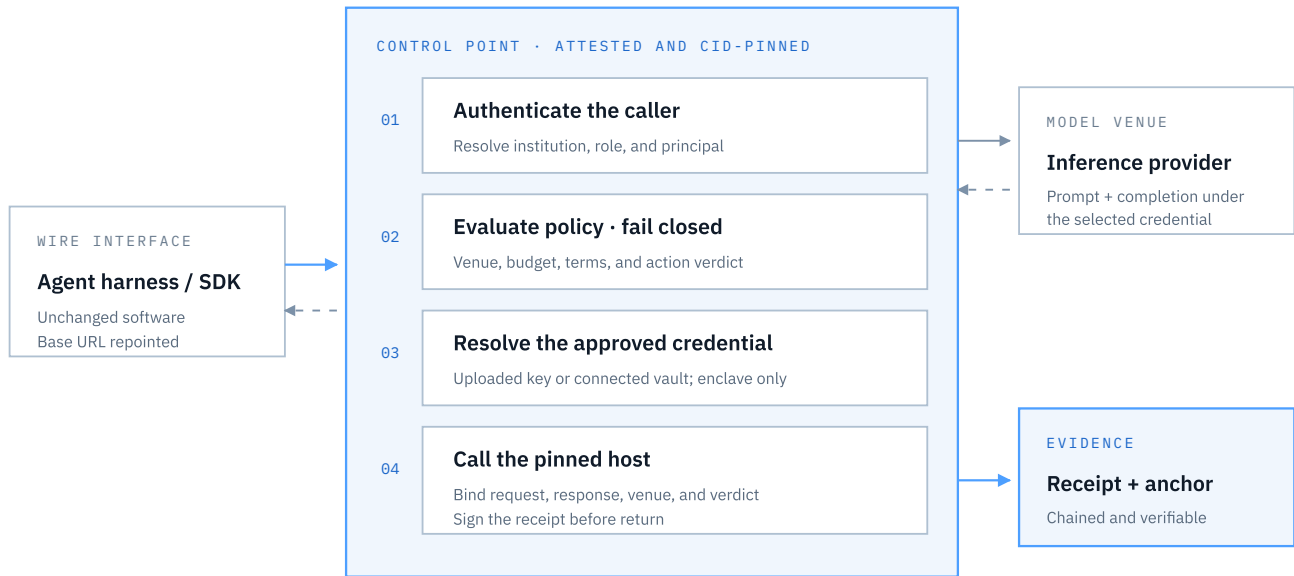
The control point speaks the two wire formats the ecosystem already standardized on. Software that talks to Anthropic's API is repointed with one environment variable; software using the OpenAI-compatible format (most open harnesses and custom enterprise loops) is repointed the same way:

```
# Endpoint and key are issued during onboarding.  
export ANTHROPIC_BASE_URL=https://<ember>/ember/anthropic  
export ANTHROPIC_API_KEY=<ember key>
```

Nothing else changes. A coding agent, an internal automation, or a vendor's custom multi-model harness continues to run exactly as before; its model calls now pass through the attested control point. The vendor's prompt assembly, retrieval, and agent steps remain on its infrastructure until the clean-room profile of Section 10 re-houses them inside the enclave. This is deliberately the smallest possible first step: a pilot begins here, measures, and then grows into full Sovereign Mode.

6.2 The call path

FIG. 3 The control point call path. A model call enters on a standard wire format and leaves only after authentication, a fail-closed policy verdict, and in-enclave credential resolution. The receipt binds the request, response, venue, credential fingerprint, and policy verdict before the response returns.



6.3 Credential choice is a policy decision

Data authority and inference billing are independent. Policy selects a credential controlled by the institution or by the vendor. In the uploaded-key path, a user supplies a provider API key through the product; Ember immediately seals it to the approved workload and never stores it as plaintext application configuration. In the connected-vault path, an organization references a credential in its existing secret manager through an approved connector. In both paths, plaintext is exposed only inside attested memory to the exact CID-pinned code authorized to make the call.

- **Institution-owned credential.** The institution's provider agreement, pricing, and retention terms govern the call.
- **Vendor-owned credential.** The vendor funds inference under its own agreement while the institution continues to govern which provider, endpoint, terms class, and data class are allowed. The receipt proves that this relationship was selected; it does not prove the provider honored the agreement.
- **The source is evidence.** The receipt records the credential-owner class and a full-length fingerprint of the credential actually used, without revealing the secret or the vault location.
- **Cross-venue use is refused.** A credential bound to one provider cannot be exfiltrated by dispatching it to another endpoint.

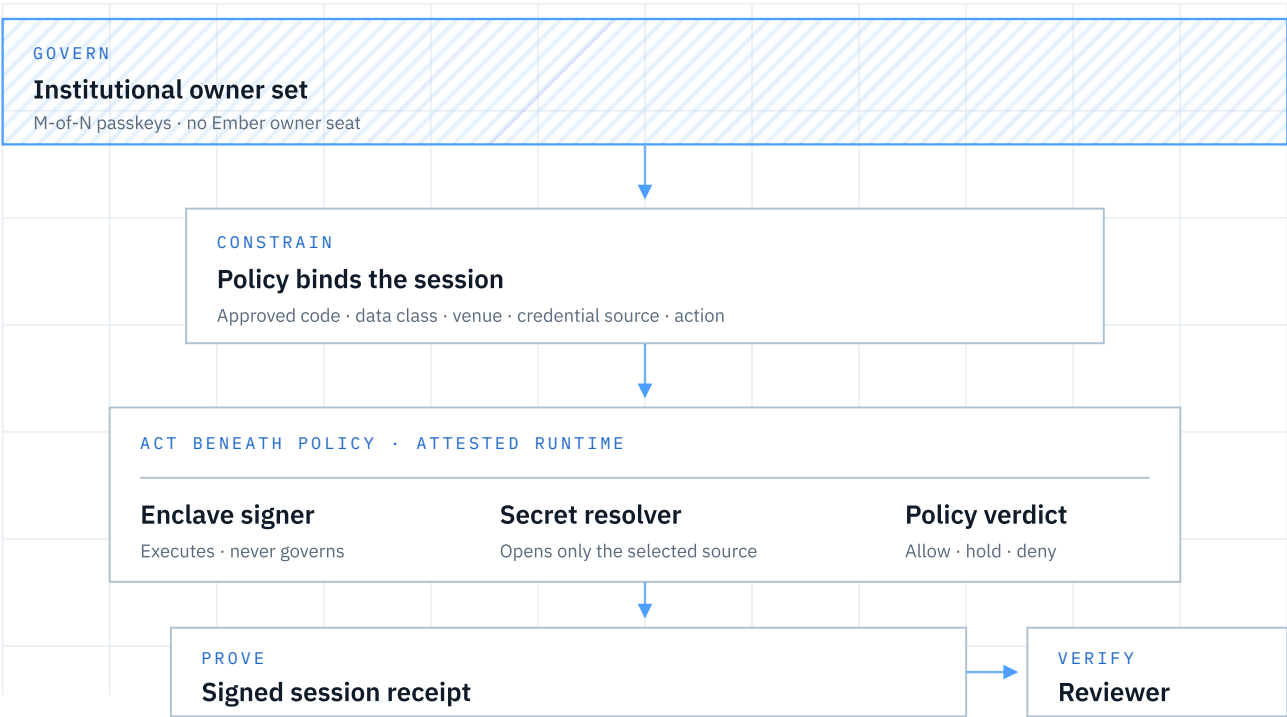
6.4 The venue gate

Egress is fail-closed. The router calls only a policy-approved tuple of provider, endpoint, credential reference, and terms class; the exact upstream host and credential fingerprint are bound into the receipt. If policy evaluation or credential resolution cannot complete, the call does not leave. An institution proves which venues its data reaches, which commercial relationship authenticated each call, and which agents or data classes were in scope.

The institution governs the complete session.

The owner set governs policy. Policy constrains the enclave. The receipt proves how that authority shaped each model call and tool action.

AUTHORITY FLOWS DOWN · EVIDENCE FLOWS OUT



7. Identity, authority, and credentials

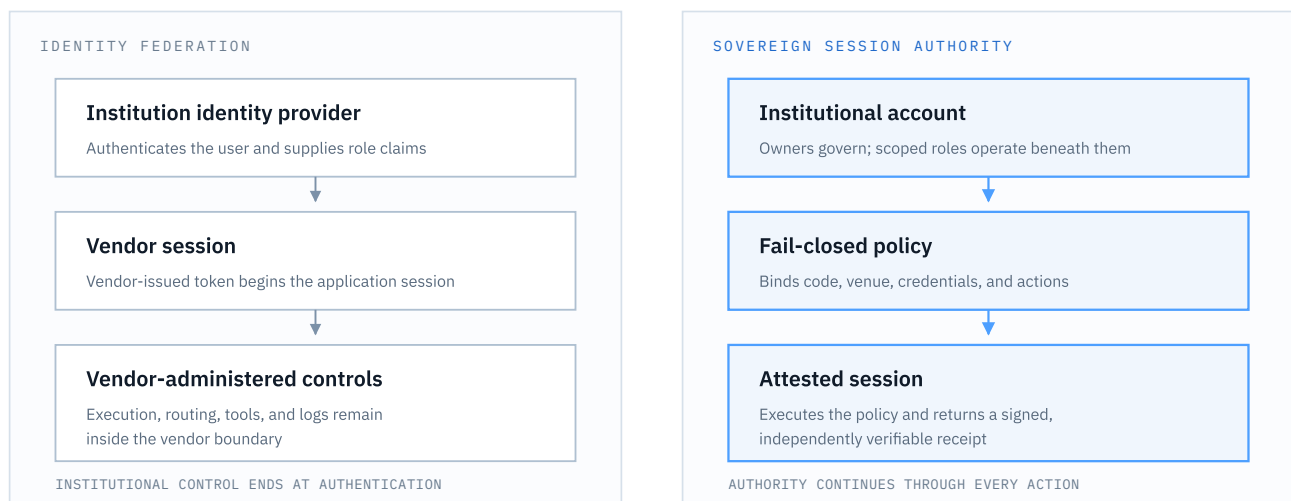
Sovereign Mode separates three forms of authority. Identity credentials establish who is acting. Governance credentials determine who changes policy or approves code. Provider credentials authenticate an inference call and determine whose provider terms apply. The session receipt binds the relationship among all three.

7.1 Enterprise SSO stops at the session boundary

Enterprise SSO is the closest familiar pattern: an institution supplies federated identity to a vendor rather than accepting a vendor-local account.^[5] SSO can authenticate a user, carry role claims, and revoke future access. By itself, however, it does not attest which vendor code handled the resulting session, constrain which inference provider received data, or independently govern the authority of an agent after login. Those controls return to vendor-administered code and vendor-held logs.

Sovereign Mode composes with SSO rather than replacing it. The institution's identity becomes an input to an authority chain that continues through policy evaluation, attested execution, and a receipt. Authentication answers who entered; session authority answers what the resulting session was permitted to do and how an independent reviewer proves it.

FIG. 4 Identity versus session authority. Conventional SSO gives an institution control over authentication, while execution and evidence remain inside the vendor boundary. Sovereign Mode extends institutional authority through policy into the session and returns independently verifiable evidence.

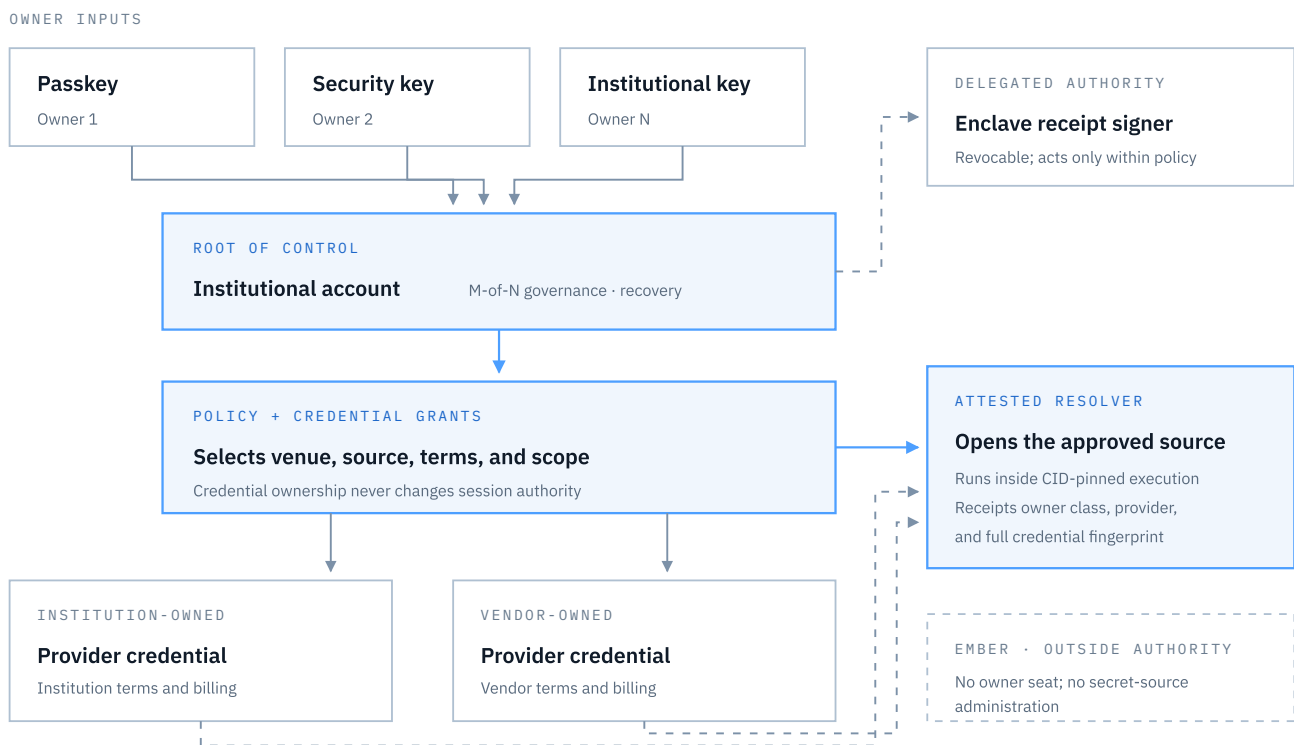


7.2 The institution's root: a passkey-controlled account

An organization's root of control is a smart contract account (SCA) on the ledger, created and owned through a WebAuthn passkey ceremony.^[2] The ceremony uses the same public-key credential model institutions already deploy through platform authenticators and security keys. Signup mints the account; no seed phrases, wallet software, or token are required. Receipts bind to an account governed by the owner set.

- **1-of-1 is the floor.** A solo developer’s account with a single passkey is a complete, legitimate organization.
- **M-of-N is growth.** An enterprise expands the owner set: multiple passkeys, security keys, or existing institutional keys, with a quorum required for governing actions. The owner set is the control list for everything below it.
- **Roles are scoped.** Human roles (administrator, operator, approver, viewer) govern; agent and API credentials resolve to a deliberately leashed role that can act within policy but can never govern, loosen policy, or move keys.

FIG. 5 The authority and credential hierarchy. Institutional owners govern policy and install a revocable enclave signer beneath it. Policy selects a provider credential owned by either party; the attested resolver opens only its approved sealed or connected source and receipts the owner class and fingerprint. Credential ownership never moves institutional authority.



7.3 Secret resolution and sealing

A credential source is registered as a policy-addressable reference, not copied into application configuration. When the source supports attested workload release, the secret manager grants the pinned runtime scoped, short-lived access to the selected credential. Otherwise the credential is sealed under the organization’s chain-secured group with a decryption condition naming the approved CID. Both paths have the same security invariant: the plaintext credential is available only to the approved code inside attested memory, for the duration of the call.

Rotation and revocation remain with the credential owner. A connected-vault rotation happens at the source; an uploaded-key rotation replaces the sealed credential. The runtime recomputes a full-length fingerprint after

resolution and binds it, together with the owner class and provider, into the receipt. The source address and secret value remain private.

7.4 The enclave's own authority

The environment holds an enclave signer registered on the organization's account. It is installed by the owners, constrained by an owner-approved policy, and revocable by the owner set; it is how receipts get signed and how policy-authorized on-chain actions execute without any human key ever entering the enclave. This workload key is not an agent identity or a governance role. The server operating all of this holds no owner key. The owners' quorum is verified inside the enclave for governing operations, so a compromised host cannot forge a governance action (Section 11).

Properties G3 and G6 in one sentence: the institution's quorum governs the session, the enclave acts beneath that policy, an institution- or vendor-owned credential authenticates inference, and Ember has no owner or secret-source-administrator seat.

8. Policy

Policy is the institution's side of the control point: a machine-evaluated statement of what their data and their agents may do. It is administered by the organization's owners (Section 7), bound to the organization's on-chain identity, and evaluated deterministically inside the environment before anything leaves it.

8.1 What a policy expresses

- **Venues and credential sources.** An allowlist of provider, endpoint, credential-owner class, credential reference, and terms class, scoped per agent and data class. This makes the routing and commercial relationship of Section 6 durable.
- **Budgets.** Spend and usage ceilings per day and per session, enforced at evaluation time, with consumption recorded in the receipt.
- **Terms flags.** Requirements such as no-training and retention class, stamped into every receipt so the terms under which a call was allowed are part of the permanent record.
- **Tenant scope.** For vendors operating Sovereign Mode across institutions: which credential references and data classes each tenant's sessions may use, so tenants are isolated by construction rather than convention.

8.2 Verdicts

VERDICT	EFFECT
ALLOW	The call proceeds. The rule that matched and any budget consumed are named in the receipt.
ALLOW-WITH-NOTIFY	The call proceeds and a notification is raised to the organization's operators.
HOLD	The call pauses pending human approval: a designated approver co-signs with their passkey, and the approval becomes part of the record. Built for high-stakes agent actions.
DENY	The call never leaves the environment. Denials are receipted too; a reviewer can see what was refused and why.

Evaluation is fail-closed (Section 6.4): no verdict, no egress. Because verdicts are deterministic functions of the policy and the request, a reviewer can replay any receipt against the policy that was in force and confirm the verdict independently.

The same machinery generalizes beyond model calls. The action vocabulary is extensible to tool calls, so an agent's individual actions (send, transfer, deploy) can be gated by the same verdicts, with `hold` routing to a human passkey. Section 12 distinguishes deployments that govern model routing from those that also govern tool authority.

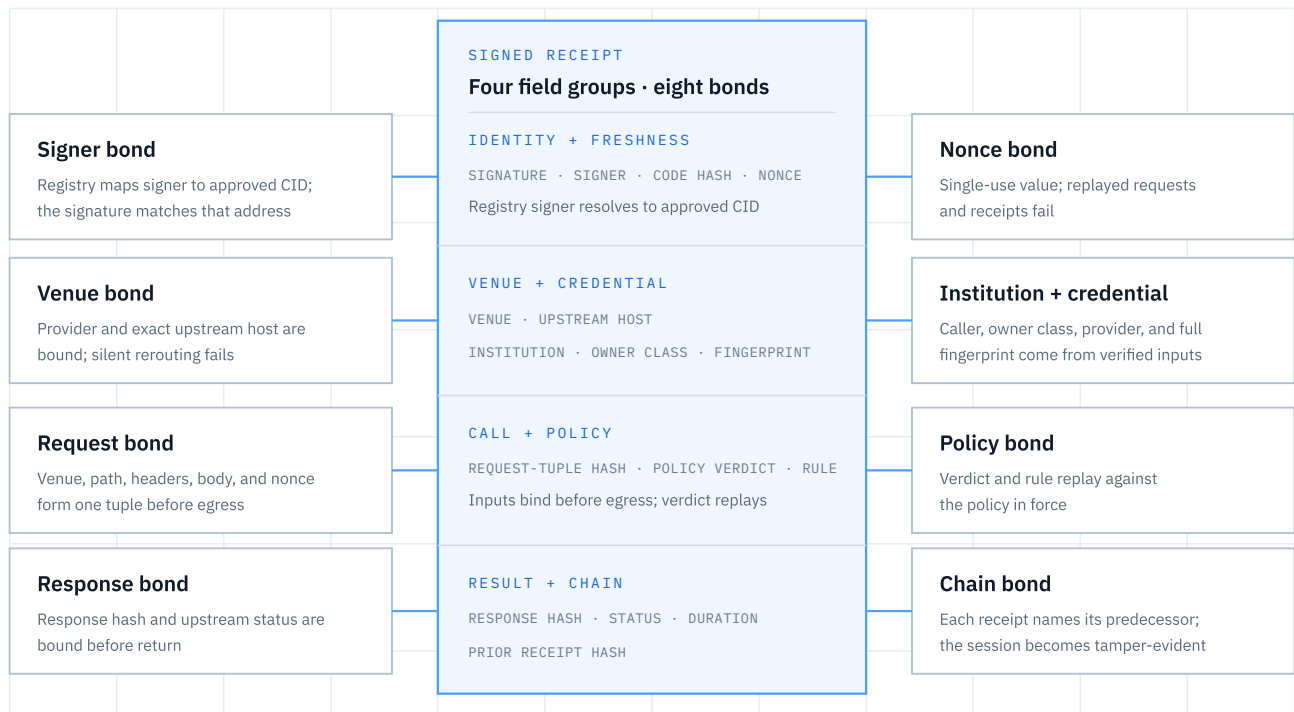
9. Receipts

A receipt is the system’s unit of evidence: a signed statement, produced inside the attested environment, of what a call or session actually did. Receipts contain hashes and metadata, never content. They serve three roles at once: the security reviewer’s continuous audit, the institution’s usage record, and the billing meter.

9.1 What a receipt binds

A receipt is only accepted as valid if **every** bond below holds. The rule: each field a verifier relies on is either signed hardware state, recomputed inside the enclave, or resolved from the ledger; nothing load-bearing is self-declared.

FIG. 6 The receipt and its bonds. Verification requires all bonds simultaneously; weakening any one of them re-opens a specific attack (self-declared signer, silent rerouting, replay, cross-org use, truncated fingerprints, edited history).



9.2 Signing and canonicalization

Receipts are signed inside the enclave by the enclave signer of Section 7.4, over a canonical JSON serialization conforming to the JCS model.^[4] Two independent implementations therefore hash the same receipt to the same bytes. The signer’s address is resolved from the on-chain registry entry for the running code’s CID; a receipt signed by any other key, including one that merely claims the right CID, fails verification.

9.3 Chaining and anchoring

Within a session, each receipt carries the hash of its predecessor, forming a chain: removing, inserting, or reordering a call breaks it visibly. Chains are anchored to the ledger in Merkle batches, which bounds the window in which even the log's operator could rewrite anything, and makes the durable record one no party, Ember included, can quietly edit. Anchoring cadence is an economic parameter, not a security assumption: the chain structure alone makes tampering evident between anchors.

9.4 Independent verification

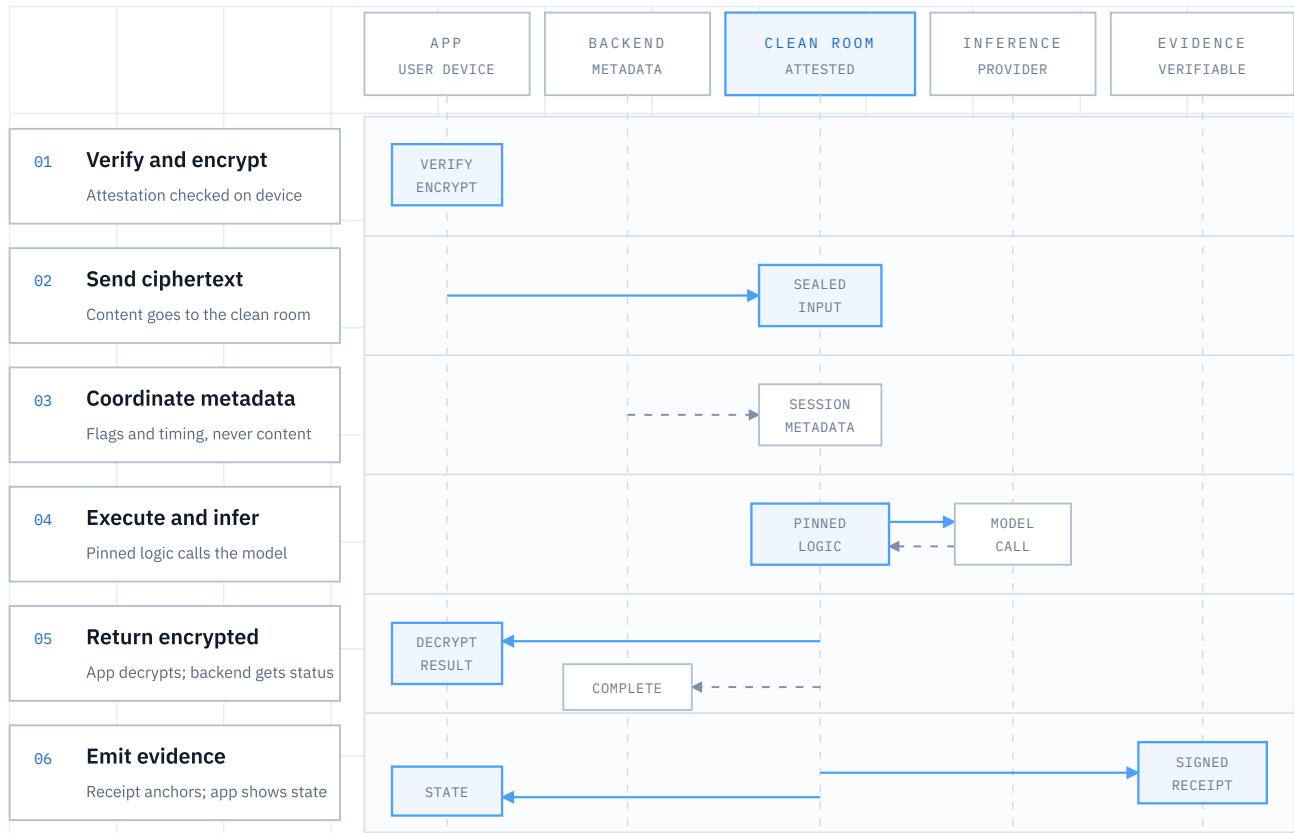
Verification is a published procedure over verifier-supplied inputs: the receipt, the ledger's registry entries, and the attestation. The institution-facing client receives each receipt body at session time; the institution's audit estate may retain it, while the ledger stores only its anchor. An open-source verifier walks a chain and checks every bond; an institution's security team can run it, an auditor can run it, and neither needs an account with Ember or an operator log. The intended consumption is practical: a reviewer can place the receipt and verification output directly into the security-review record.

10. Sovereign Mode: a full session

The preceding sections defined the parts. This section runs one session through all of them: an end user inside the vendor's product, working on material their employer would never send to a vendor in plaintext.

1. **The user turns Sovereign Mode on.** The Ember SDK inside the vendor's app fetches and verifies the environment's attestation (Section 5.2), then encrypts the user's content, text and files alike, on the device, to the verified environment.
2. **Ciphertext travels; metadata coordinates.** The encrypted payload goes to the clean room. The vendor's backend receives what it needs to run a product (session identifiers, timing, feature flags) and nothing it could read.
3. **The vendor's logic runs, pinned.** Inside the environment, the vendor's own prompt assembly, document processing, retrieval, and agent steps execute, the exact CID-approved build. This is the load-bearing move of the whole architecture: the vendor's product logic is not re-implemented by Ember, it is re-housed.
4. **Inference happens per policy.** Model calls exit through the control point: policy verdict, approved institution or vendor credential, pinned venue, receipt per call (Section 6.2).
5. **The result returns encrypted.** Only the user's app can decrypt it. The vendor's backend learns that the request completed, and its cost.
6. **The session leaves a receipt.** Content hashes, code hash, venues, verdicts, and duration, chained and anchored (Section 9). The app shows the sovereignty state and a verification link the user's security team can follow.

FIG. 7 One session, precisely. The whole product experience happens inside the vendor's app; content is readable in exactly two places, the user's device and the attested room. The vendor's backend coordinates on metadata and never touches plaintext.



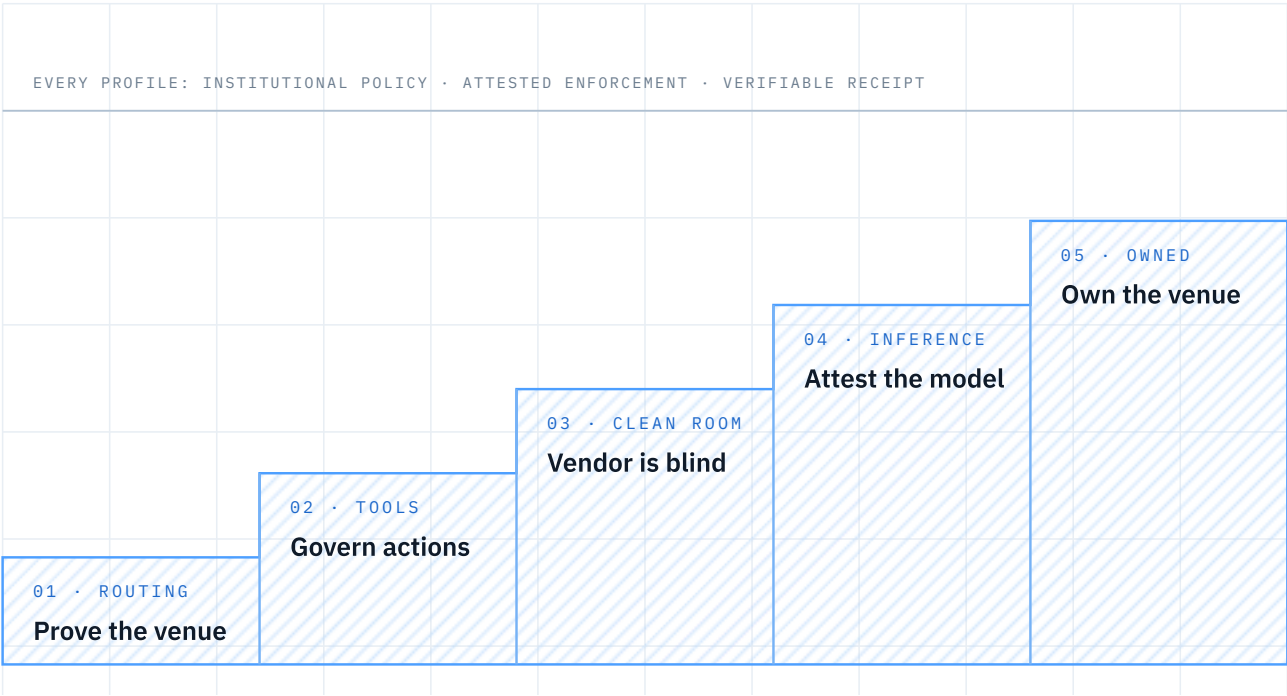
10.1 What the vendor gains, structurally

The inversion is worth stating from the vendor's side because it converts a liability into a product property. The vendor still serves the product: its prompts, retrieval, agent logic, brand, and commercial relationship. It may also supply the inference credential. But the vendor's infrastructure cannot disclose, lose, or be compelled to produce institutional content it never possesses. Instead of re-auditing operational discipline release after release, the reviewer verifies the approved build and lets each session re-prove the relevant properties.

The assurance profile matches the workload.

One integration spans verified routing through institution-owned infrastructure. Each receipt names the profile the session actually satisfied.

FIVE ASSURANCE PROFILES · ONE AUTHORITY AND EVIDENCE MODEL



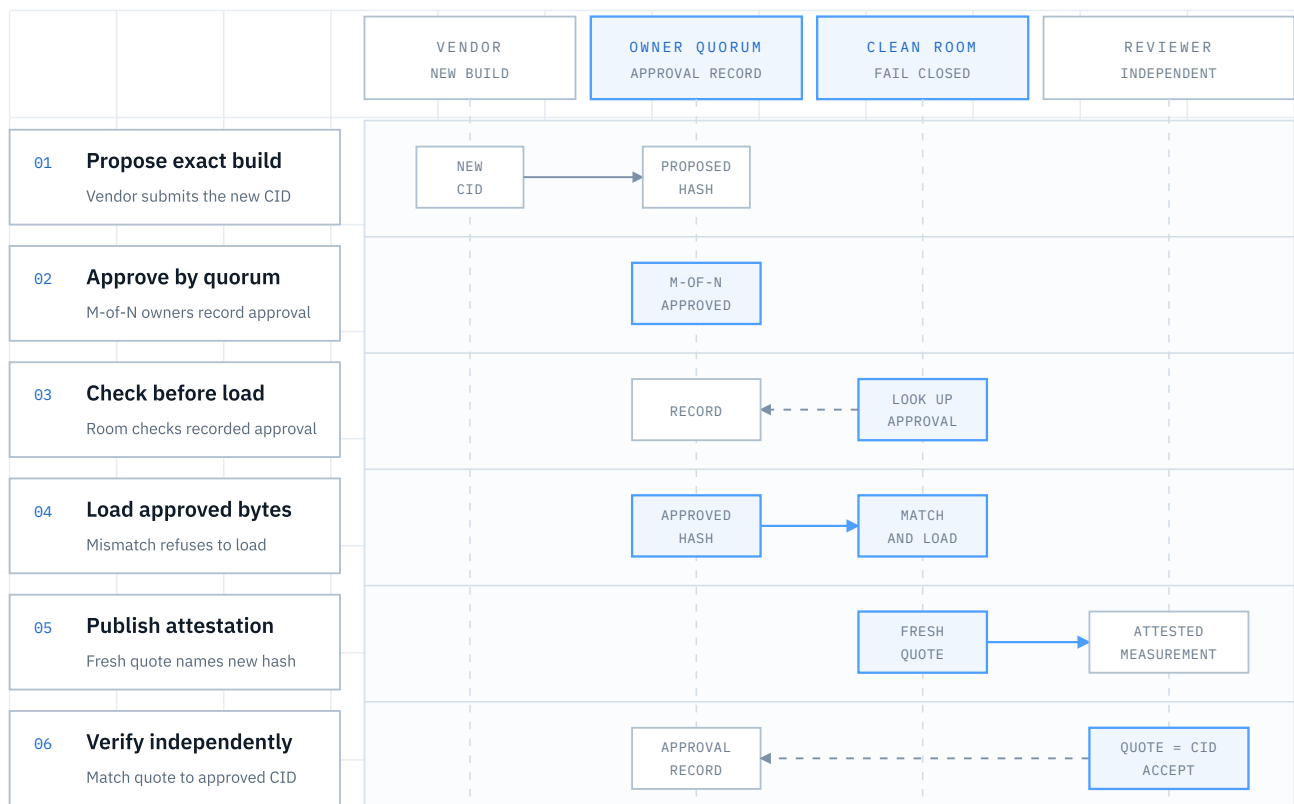
11. Governance and upgrades

The most dangerous operation in any attested system is changing the code, because a malicious upgrade (adversary A4) inherits every key and every promise made about the old build. The system therefore treats upgrades as governed, auditable events rather than deployments.

11.1 The upgrade gate

A new build enters governance by its CID. An on-chain multisig, with a quorum of M-of-N owners appropriate to the deployment, approves the CID before any environment loads it. The environment checks the ledger before loading code; an unapproved measurement refuses to boot. After an approved upgrade, the environment's fresh attestation quotes the new hash, and any institution or auditor confirms that the attestation matches the on-chain approval. No vendor release key and no Ember employee pushes code past this gate.

FIG. 8 The upgrade sequence. New code reaches the clean room only through on-chain M-of-N approval; the room refuses unapproved builds, and the institution can check every attestation against the approval record.



11.2 Tenant organizations and the claim

A vendor offering Sovereign Mode provisions one organization (Section 7) per institution: its owner set, policy, approved credential references, and receipt history, isolated by construction. The organization has two administrative states, and the state is always disclosed because the two make different governance claims:

- **Vendor-administered.** The vendor’s application provisions and operates the tenant organization. The honest claim is “the vendor administers these controls, provably”: credential references are scoped per tenant and receipts accrue per tenant, but governance sits with the vendor.
- **Institution-held.** A control in the vendor’s product starts the institution’s passkey ceremony on the organization that already holds its policy and receipts. Institutional credentials take the owner seat; the vendor application keeps a leashed agent credential and continues operating the product. Nothing migrates: the institution now governs venues, credential sources, and receipts directly.

The claim is a governance move, not a data or credential migration, and it is deliberately vendor-surfaced. Ember holds no owner seat at either state, so no Ember-side path exists to eject a vendor from its relationship with an institution or to market to a vendor’s tenants. Governance moves from the vendor to the institution; the provider credential may remain vendor-owned, become institution-owned, or vary by policy. Alongside the embedded receipts view, an institution may use its own estate surface for cross-vendor receipts, policy administration, and recovery.

12. Deployment models and assurance

An institution choosing an AI deployment is choosing both an operating model and a trust model. Vendor SaaS centralizes operation and trust. Traditional BYOC moves operation into the institution's perimeter but leaves broad trust in application code and release processes. Sovereign Mode supplies a third structure: the workload remains operated as a service while its data path, code identity, authority, and evidence are independently verifiable.

DIMENSION	VENDOR SAAS	TRADITIONAL BYOC	SOVEREIGN MODE
END-USER EXPERIENCE	Unchanged. The product as shipped.	Lagged. The same product, behind internal deployment.	Unchanged. Sovereignty is a toggle.
OPERATING MODEL	Vendor-operated. The institution consumes the service.	Institution-operated. The platform team deploys and maintains the vendor stack.	Selectable. Hosted, institution cloud, or institution-owned hardware under one verification model.
DATA EXPOSURE	High. Plaintext in the vendor's backend.	Contained. Plaintext in memory inside the buyer's perimeter.	Minimal. Plaintext only inside the attested enclave.
USAGE GOVERNANCE	Contractual. MSAs and DPAs.	Configurational. Cloud IAM, maintained by the buyer.	Programmable. Policy enforced in code, receipted per session.
INSTITUTIONAL BURDEN	Low. Standard SaaS operations.	High. Tenancy, egress, upgrades, and capacity.	Profile-dependent. Managed third-party compute through owner-operated hardware.

12.1 Structural comparison with hardened BYOC

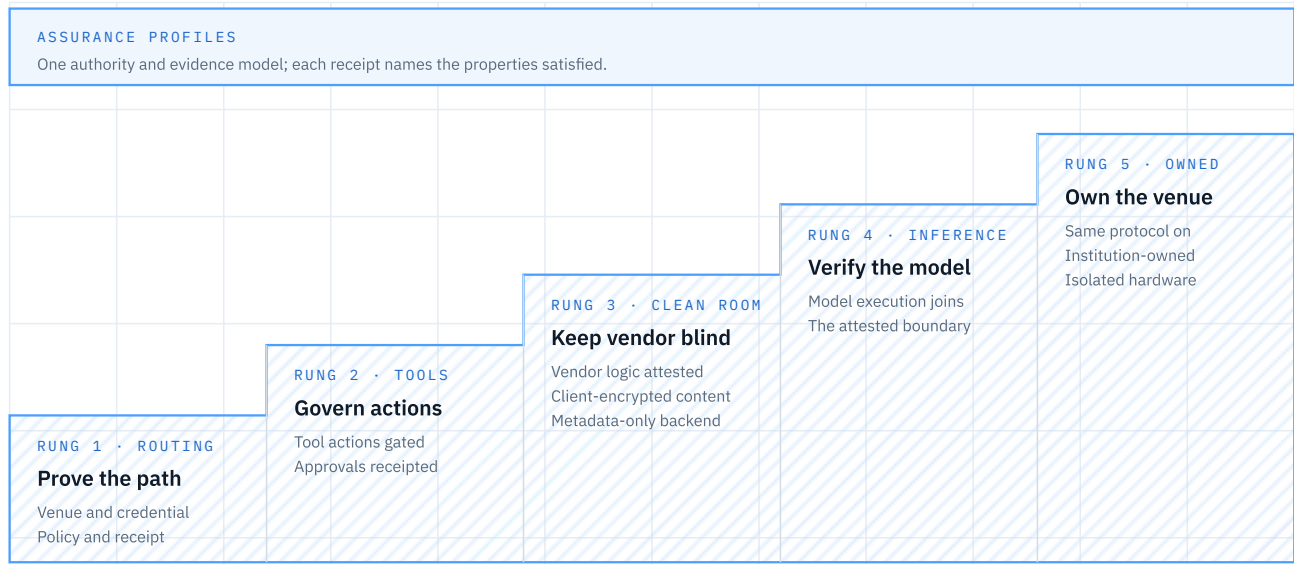
VECTOR	TRADITIONAL BYOC	SOVEREIGN MODE
TRUST ANCHOR	Operational. Network perimeter, IAM policy, the vendor’s release pipeline.	Cryptographic. Silicon attestation chain and a content-pinned, audited build.
DATA IN USE	Exposed. Plaintext in application memory throughout processing.	Encrypted in use. Plaintext exists only inside the attested enclave.
SESSION AUTHORITY	Perimeter-derived. Institution IAM protects the deployment; vendor code enforces application actions.	Institution-governed. Owner-approved policy is evaluated inside the attested session.
PROVIDER CREDENTIAL	Deployment-specific. Usually stored in institution KMS; exposed to the application at runtime.	Party-selectable. Institution- or vendor-owned; uploaded and sealed or connected from a vault; owner class, provider, and fingerprint receipted.
VENDOR ACCESS PATH	Governed. Support roles and update pipeline, under policy and exceptions.	Absent. Restored only by institution-key grant, receipted on the record.
AUDIT TRAIL	Mutable. Centralized logs held by the operating parties.	Immutable. Signed receipts chained and anchored to the append-only ledger.
REVIEW ARTIFACT	Recurring audit. Architecture re-reviewed for every release.	Verifiable proof. Attestation checked against the audited build, per session.
DEPLOYMENT BURDEN	High. Tenancy, egress control, upgrades land on the buyer’s platform team.	Low. Venue-independent: hosted, institution cloud, or institution hardware.
RESIDUAL TRUST	Broad. Vendor organization, cloud operator, sustained egress discipline.	Narrow, named. Silicon attestation roots and the audited enclave code.

A hardened BYOC deployment is real security work, and the comparison credits it. The structural difference is what remains to be trusted and whether an institution can verify that trust per session. The models also compose: the attested environment can run inside the institution’s cloud account or on hardware it owns.

12.2 The assurance continuum

Palantir’s *Institutional Sovereignty in the Age of AI* frames the compute decision as a workload-specific ladder: owned and isolated hardware for the most sensitive work, verified third-party compute where ownership is unnecessary, and contractual controls for lower-risk usage.^[1] This paper adopts that reasoning and extends it to the complete AI session. The first two rungs are control-point entry profiles; full Sovereign Mode begins at the clean-room rung. Rungs 1–4 broaden what is governed or kept confidential. Rung 5 changes who owns the venue and composes with rung 3 or rung 4; it is shown last because it adds physical and administrative control, not because owned hardware depends on attested inference. None is a release status, and none permits a claim above what its receipt proves.

FIG. 9 The assurance profiles. Routing and tools are entry profiles; the clean room is full Sovereign Mode. Attested inference adds model confidentiality. An owner-operated venue composes with the clean room or attested inference; physical ownership is not a substitute for attestation or policy.



RUNG	PROPERTY EARNED	RESIDUAL TRUST
1 • ROUTING	Provider routing and credential use are provable. Policy selects the venue and institution- or vendor-owned credential; each call is receipted.	The router and external inference provider see prompt plaintext transiently. Provider terms govern its use.
2 • TOOLS	Agent authority is governed. Individual tool actions are policy-gated and receipted; high-stakes actions can require institutional approval.	Content confidentiality remains that of rung 1.
3 • CLEAN ROOM	The vendor serves what its servers cannot read. Client-encrypted content and pinned vendor logic execute inside the attested boundary.	An external inference provider still reads prompts under the selected credential's terms.
4 • INFERENCE	The inference infrastructure operator cannot read prompts or outputs. Model execution is itself attested and confidential.	Silicon roots, approved code, and the selected model artifact remain trusted.
5 • OWNER-OPERATED	The institution also controls the physical and administrative venue. The same attested protocol runs on isolated hardware it owns.	Endpoint security, silicon behavior, approved code, and institutional operations remain in the trusted base.

13. Operational model

The method separates invariant properties from measurements and commercial terms versioned outside the architecture. Latency targets, capacity, and price change; the meter, evidence path, and separation of inference billing do not.

13.1 Performance is measured per profile

Attestation, policy evaluation, credential resolution, model inference, and receipt finalization are distinct intervals. The receipt records attested execution duration, while ordinary telemetry reports transport and provider latency. Performance claims identify the assurance profile, hardware profile, model, payload shape, and streaming mode measured. This prevents a benchmark from one profile becoming an architectural promise for another.

The runtime streams natively from the attested boundary or replays a completed response over a standard event stream. Both modes preserve the same invariant: the response hash and upstream status are bound before the final receipt verifies.

13.2 The meter is evidence

Attested compute duration is the unit the execution layer proves. Token counts are carried for transparency, but they are reported by the inference venue rather than measured by the control plane. Commercial pricing changes without changing the receipt schema or the security model.

13.3 Inference remains on the selected provider relationship

Provider charges flow through the credential selected by policy. If the institution supplies the credential, its negotiated pricing and terms apply. If the vendor supplies it, the vendor's agreement applies. Ember does not re-intermediate the inference charge; its economic role is the attested execution and evidence layer. This separation lets a vendor package Sovereign Mode as part of its own institutional offering without obscuring whose provider terms governed a session.

14. Boundaries and open problems

These boundaries define the method; they are not a development report. Every deployment declares its assurance rung and the choices below. A stronger profile changes a named property; it does not retroactively strengthen sessions that ran under a lower profile.

-
- | | |
|----|---|
| B1 | External inference. At rungs 1–3, an external inference provider reads prompts under the terms of the institution- or vendor-owned credential selected by policy, preserving access to any provider permitted by that policy. Rung 4 moves model execution into attested confidential compute and therefore depends on a model artifact available in an approved confidential venue. Rung 5 moves an applicable attested venue onto institution-owned hardware and composes with rung 3 or rung 4. |
|----|---|
-
- | | |
|----|---|
| B2 | Client integrity. The baseline clean-room profile trusts the vendor-authored client to verify attestation and encrypt honestly. An isolated trusted component, reproducible client build, browser integrity mechanism, or institution-controlled client can narrow that trust. The receipt must identify which client profile initiated the session. |
|----|---|
-
- | | |
|----|---|
| B3 | Secret-source integrity. Connected-vault administrators and authorized uploaded-key roles can rotate or replace a credential. Policy allows a mutable reference, pins a fingerprint, or approves a fingerprint set. The runtime fails closed when the resolved provider, owner class, or fingerprint does not match policy, and the receipt makes every authorized rotation visible. |
|----|---|
-
- | | |
|----|--|
| B4 | TEE side channels and hardware errata. Confidential-computing platforms have a continuing history of timing, cache, speculative-execution, and firmware issues. Firmware and platform state belong in attestation; policy can exclude measurements or platforms. Owner-operated hardware reduces operator exposure but does not eliminate silicon risk. |
|----|--|
-
- | | |
|----|--|
| B5 | Ingress termination. An attested router can protect credentials and receipts while transport terminates at a relay, which does not by itself make prompt ingress confidential. The clean-room profile obtains content confidentiality from client-side encryption. Enclave-terminated transport or a direct attested session narrows the lower-rung exposure. |
|----|--|
-
- | | |
|----|--|
| B6 | Receipt anchoring. Receipts are signed and chained immediately; ledger anchoring may be batched. The maximum unanchored window is a deployment parameter that must be disclosed. Shorter cadence reduces the interval during which a log operator could omit an otherwise valid suffix. |
|----|--|
-
- | | |
|----|--|
| B7 | Availability and streaming. Confidentiality, integrity, and evidence do not guarantee availability. Provider outages, enclave capacity, denial of service, and first-token latency remain operational concerns. Streaming implementations must preserve response binding even when delivery is incremental. |
|----|--|
-
- | | |
|----|---|
| B8 | Endpoints, models, and approved code. Content is readable on the user’s endpoint by construction. Attestation proves which code and model artifact ran, not that they are correct or safe. Endpoint security, code review, model evaluation, and the institution’s approval process remain in the trusted base at every profile. |
|----|---|
-
- | | |
|----|---|
| B9 | Commitment privacy. Receipt bodies contain content commitments, not content, and are returned to the institution rather than published on the ledger. A plain hash of low-entropy input can still permit confirmation guesses by anyone given that receipt. Each deployment must specify whether commitments are salted or keyed and who is able to verify them. |
|----|---|
-

The owner-operated venue is the top of the infrastructure ladder, not the end of trust. It replaces third-party physical and administrative control with institutional control while preserving the need to verify silicon, code, model artifacts, policy, and receipts.

15. Conclusion

The missing control in institutional AI is not authentication at the login screen. It is durable authority over what happens after authentication: which code can touch data, which provider and credential may be used, which actions an agent may take, and what evidence remains. Enterprise SSO establishes identity; Sovereign Mode carries institutional authority through the session.

The composition is the contribution. Attested hardware narrows infrastructure trust. Content addressing pins the reviewed code. Client-side encryption removes the vendor backend from the content path. Institution-governed policy selects the venue and the institution- or vendor-owned inference credential. Signed receipts turn each session into evidence a third party verifies without relying on an operator's account of events.

No single deployment profile maximizes every dimension. Institutions match assurance to workload sensitivity, from verified routing through attested third-party compute to an owner-operated venue. The integration and authority model remain constant as the venue changes. The vendor keeps its product and relationship. The institution keeps authority over its data and sessions. The reviewer receives the proof.

Appendix A — Verifying an attestation, step by step

The procedure a security reviewer runs before approving the first payload, expanded from Section 5.2. Nothing in it requires an account with the vendor or Ember.

1. **Fetch the quote.** The deployment's verification interface returns the enclave attestation document: hardware evidence, the measurement of the running build, relevant platform state, and a freshness value. Transport and endpoint naming are deployment-specific; the evidence fields are not.
2. **Verify hardware authenticity.** Validate the quote's certificate chain against the silicon vendor's published root of trust, exactly as for any confidential-computing deployment. A passing check proves a genuine enclave on patched silicon produced this quote; an emulator or modified host cannot.
3. **Verify code identity.** Compare the quoted measurement against the CID of the build you audited (or that your auditor signed off). The CID is also on the ledger with its M-of-N approval (Section 11.1); confirm the approval exists and predates the session.
4. **Verify the session's receipts.** Run the verifier over the session's receipt chain. It checks every bond of Section 9.1 against the verification inputs: registry-resolved signer, venue and host, request and response hashes, nonces, institution identity, credential-owner class and fingerprint, and chain continuity. It flags drift if any quoted measurement diverges from the approved CID.

Steps 1–3 are performed once per build; step 4 is continuous and cheap. The output of all four is forwardable and attaches directly to a security-review ticket as evidence rather than being summarized in prose.

Appendix B — Receipt field reference

FIELD	CONTENT	VERIFIED AGAINST
<code>signature</code>	Signature over the canonicalized receipt, produced inside the enclave.	The signer address the on-chain registry maps to the running code's CID.
<code>code</code>	CID of the build that produced the receipt.	The ledger's approval record; the attestation quote's measurement.
<code>venue</code>	Provider identity and the exact upstream host called.	The policy's approved venue tuple and the resolved credential's provider binding.
<code>request</code>	Hash of the request tuple: venue, path, headers, body, nonce.	Recomputed by the verifier from the tuple; enclave refuses on mismatch pre-egress.
<code>response</code>	Hash of the response body and the upstream status code.	Recomputed from the response delivered to the caller.
<code>nonce</code>	Single-use value bound into the request tuple.	Uniqueness across the organization's history; replay fails.
<code>institution</code>	The governed organization whose policy authorized the session.	Equality with the authenticated caller's institutional context.
<code>credential_owner</code>	Institution or vendor: the class of party supplying the provider credential.	The policy-approved credential reference and secret-source grant.
<code>credential</code>	Full-length fingerprint of the provider credential actually used.	Recomputed inside the enclave from the resolved secret, never trusted from storage.
<code>policy</code>	Verdict, the rule that produced it, budget consumed, terms flags.	Deterministic re-evaluation against the policy in force at the time.
<code>duration</code>	Attested compute time for the call or session; the billing meter.	Plausibility and cross-checks along the chain.
<code>prior</code>	Hash of the preceding receipt in the session.	Chain continuity; anchored in Merkle batches to the ledger.

Appendix C — Glossary

- Agent harness.** Third-party software that drives model calls: an AI SDK, a coding agent, or a vendor's custom loop. Integrates with the control point by base URL alone.
- Attestation.** A hardware-signed statement of exactly what code and configuration is running inside an enclave, verifiable against the silicon vendor's published root of trust.
- BYOK.** Bring your own key. The credential profile in which the institution supplies the inference-provider credential. Vendor-supplied credentials use the same attested resolver and receipt bonds.
- CID.** Content identifier: a cryptographic hash naming an exact build. The unit of code identity, approval, and audit.
- Clean room.** The sealed, attested execution environment in which the vendor's application logic runs on decrypted institutional content without persisting plaintext session state.
- Control point.** The attested model-call router: authentication, policy, credential resolution, venue enforcement, and receipts at the model-call boundary.
- Credential source.** A policy-addressable provider credential in an institution- or vendor-controlled secret manager, or a credential sealed directly to approved attested code.
- Ember SDK.** The vendor-embedded component that verifies attestation, encrypts client-side, and renders sovereignty state and receipts in the vendor's product.
- Estate surface.** The institution's own console: receipts across vendors, policy administration, key rotation, recovery.
- Ledger.** The public, append-only record carrying code approvals, organizational addresses, and receipt anchors. Only hashes, approvals, addresses, and anchors are public; content, secrets, and receipt bodies are not. It is the one log no operator can quietly edit; there is no token and nothing to trade.
- Receipt.** A signed, chained, independently verifiable record of a call or session: hashes and metadata, never content.
- Sealing.** Encrypting a secret so that only a specific CID-pinned program, executing inside attested hardware, can decrypt it.
- Institution.** The governed organization whose data, policy, and audit obligations are at issue. Often the vendor's enterprise buyer.
- Smart contract account (SCA).** The on-chain account, controlled by passkeys with an M-of-N owner set, that serves as the institution's root of governance.
- Sovereign Mode.** The method defined in this paper: the vendor's product serving content it cannot read, verifiably.
- TEE.** Trusted execution environment: hardware-isolated, memory-encrypted execution with remote attestation.

Vendor. The organization that builds and serves the AI product into which Sovereign Mode is integrated.

Venue. A policy-approved execution or inference destination. For inference the durable tuple includes provider, endpoint, credential reference, credential-owner class, and terms class.

Appendix D — Selected references

[1] Palantir Technologies, *Institutional Sovereignty in the Age of AI*, Section IX, “Verify compute you don’t own,” 2026. [Source paper](#).

[2] World Wide Web Consortium, *Web Authentication: An API for Accessing Public Key Credentials*, Level 3. [W3C specification](#).

[3] IETF RFC 9334, *Remote ATtestation procedureS (RATS) Architecture*. [RFC Editor](#).

[4] IETF RFC 8785, *JSON Canonicalization Scheme (JCS)*. [RFC Editor](#).

[5] National Institute of Standards and Technology, *Digital Identity Guidelines: Federation and Assertions*, SP 800-63C-4. [NIST publication](#).
